A Python Interpreter in the Weights: General-Purpose Code Execution Inside Qwen3.5-4B, Its Cost, and the Window That Bounds It

Nikita Tolkachev

November 2026

Abstract

Qwenputer (Agrawal, 2026) compiles a 32-bit RISC-V CPU into eleven transformer layers prepended to Qwen3.5-4B, using the model's Gated DeltaNet state as RAM; the published model exposes one tool, an exact calculator, which the model invokes in its native tool-call syntax and whose response the model's own layers print. We complete the step its author lists as missing: a **Python tool** — a MicroPython interpreter that boots inside the firmware snapshot, runs the model's code in a persistent session, and returns its output, entirely within `model.generate()`. On the checkpoint itself (bf16, RTX 4090, the author's executor) the layers generated 97.3 M chip tokens over 700 model-driven items with zero mismatches against the reference machine. We measure the cost of code execution in the weights (14.8 K instructions for an empty call, 40–200 K for typical calls) and reduce it by 19–31 % with a FIRST-set parser optimization validated on MicroPython's own test suite. On ten exact-answer tasks the tool is decisive on character-level work (reverse 0 → 100 %, swap characters 38 → 100 %, count letters 78 → 100 %) and costs accuracy where the base model is already strong and the input overflows the circuit's 80-token capture window (date reasoning 84 → 54 %); mean accuracy moves from 70.8 % (base) to 83.0 %. The experiment also exposes a property of in-weights tools that an external-tool proxy cannot show: the circuit prints its response one byte per token, and a 4B model transcribing a 15-letter nonsense string from single-letter tokens garbles it 80 % of the time although the tool printed it correctly in 100 % of items; a driver-side re-tokenization of the response, which leaves the generated stream untouched, removes the loss entirely. We argue the in-weights-versus-host comparison is a control rather than a hypothesis — an exact executor cannot beat CPython — and that its value is attribution: it isolates the environment (interpreter limits, the capture window, the response's tokenization) from the model.

1 Introduction

Language models are poor at exact computation — counting letters, sorting lists, date arithmetic — and the standard remedy is a tool: the model emits a call, a runtime executes it, the result is appended to the context. Qwenputer takes the runtime away. A compiler writes the weights of a RISC-V CPU into extra transformer layers; the model's recurrent (Gated DeltaNet) state is the CPU's memory; each instruction becomes a few *trace tokens* in the model's own output; and when the model writes a `<tool_call>` the compiled layers run it and the model prints the `<tool_response>` itself. No executor, sandbox or callback is involved: the checkpoint loads as a stock `Qwen3_5ForCausalLM` and runs under `generate()`.

The published Qwenputer tool is `calc`, an exact arithmetic evaluator, and the author reports it reaches the accuracy of a host Python tool on arithmetic benchmarks. MicroPython had been run on the CPU standalone, but the model could not call it; the project's design notes list "a bridge build (P4)" as the missing piece. This paper reports that bridge, and what it reveals.

Our questions are not "does code in the weights beat a tool" — it cannot; an exact executor is at best as right as CPython. They are:

1. Does general-purpose code (an interpreter, not a purpose-built function) run exactly on the compiled circuit, driven by the model's own calls?
2. What does it cost, in instructions, trace tokens and seconds, and what dominates that cost?
3. Where the model's code is executed in the weights rather than on a host, what accuracy is lost, and why?

The answers: yes — verified first over 2.26 M tokens on the compiled circuit, then over 97.3 M tokens generated by the checkpoint itself; 15–200 K instructions per call with the parser as the largest component, reducible by a lookahead table; and, on the checkpoint, two environmental effects that a proxy benchmark could not separate: the 80-token capture window, and the byte-level tokenization of the response the circuit prints, which a driver-side re-tokenization removes.

2 Background: Qwenputer

Qwen3.5 interleaves full-attention layers with Gated DeltaNet layers, each of which carries a state matrix updated per token by the gated delta rule. With the gate held open and a one-hot key, an update overwrites exactly one row of the state; a one-hot query reads one row back. Qwenputer uses this as RAM: the CPU's registers, program counter, 32 KB of RAM, 128 KB of read-only data (DROM) and 256 KB of pre-decoded code (ROM) live in the DeltaNet heads of eleven compiled layers whose every weight is computed, not learned. The CPU is RV32IM with the Zba extension and a bignum coprocessor; the multiply-add logic is exact lookup tables in the MLPs, and the whole machine is exact in bf16. Writes arrive through the token stream, so the trace is the execution log, and the model's text behaviour is unchanged (KL to base $\approx 6 \times 10^{-4}$ by the author's measurement).

The bridge between Qwen's text and the machine (the author's "P3") captures every token between `<tool_call>` and `</tool_call>` into a text buffer: row i of the buffer holds the i-th token's UTF-8 bytes, NUL-padded to 16, for $i \leq 80$. At `</tool_call>` the circuit predicts `GO`; the firmware, parked at an `exit` system call, resumes, reads the buffer, computes, and prints: each printed byte is a `BYTE` trace token followed by its echo as the Qwen token of that byte, so Qwen reads the output as text. `HALT` parks the machine again. The 80-row limit is a property of the compiled capture circuit (its thermometer weights overflow bf16 past 80 tokens), not of any firmware.

A firmware is an ELF whose state after booting to its first park is snapshotted into the chip layers after the prompt; the boot therefore costs nothing at inference time.

3 The Python bridge

3.1 Firmware

The firmware (`main.c`, ~250 lines over the author's MicroPython port files) boots MicroPython once, then loops: park; on `GO`, flatten the capture rows into a byte string; find the `code` parameter — the literal `</parameter>` closes it, so code may contain `<`; optionally find a `data` parameter; run the code in `__main__` with REPL semantics (bare expressions print their value); print the response framing the author's `calc` uses (`\n<tool_response>\n…</tool_response>\n<think>\n\n</think>\n`); park. Three properties matter for a model-driven interpreter:

- **Persistence.** The interpreter is never re-initialised: globals, functions and imported modules survive across calls. Since a call is at most 80 tokens, this is the model's way to do more — define in one call, use in the next.
- **A hard step budget.** A hook in the bytecode loop (`MICROPY_VM_HOOK_LOOP`/`_RETURN`) counts jumps and returns; at 3,500 the firmware `longjmp`s out of the interpreter, restores the exception-handler stack, and prints `RuntimeError: step budget exceeded`. A Python-level exception would not do: user `try/except` could swallow it and keep the machine from parking, after which the conversation's circuit state would be unrecoverable. The budget bounds a call at $\approx$ 0.1–0.9 M instructions.

- **Bounded output.** 1,536 bytes per call reach the model's context; deep recursion raises `RecursionError` (MicroPython's stack check, 6 KB of stack); tracebacks are returned so the model can retry.

The `data` parameter is long plain-text input, exposed as the string global `data`. Its motivation is §6.

3.2 Interpreter features

MicroPython's BASIC feature level, which the author's port uses, has **no f-strings** (`SyntaxError`), no `random`, `re`, `datetime`, and no `str` slices with a step. A model writes all of these. Our build adds f-strings, `random` (seeded from a hash of the conversation's tool-call text, so conversations differ while every roll remains in the trace), `re`, `heapq`, `hashlib.sha256`, `math.factorial`, 3-argument `pow`, `collections.deque`, `str.partition/splitlines/center`, `bytes.hex`, and two things that required source changes: step slices for `str` (a 30-line patch to `objstrunicode.c`, unicode-safe) and `datetime` (micropython-lib's pure-Python implementation, frozen into the DROM as bytecode, with a `gmtime` shim — the machine has no clock — checked against CPython on 5,000 timestamps, and a `strftime` checked on 300). The firmware is 230 KB of the 256 KB ROM and 39 KB of the 128 KB DROM, with ~23 KB of heap.

3.3 Verification

The author's bridge test method is teacher-forced evaluation of the compiled circuit's IR over a whole session: at every position the circuit must claim ownership exactly where the reference machine does, and predict the next token wherever it owns the position. We ran it on six sessions across the firmware's revisions — an empty call; a four-call session with sorting, `str.count`, a `ZeroDivisionError` and a live call after it (348,832 tokens); `random` and `re` calls; `datetime` import and arithmetic (748,691 tokens); `[::-1]` and `strftime`; and a `data` call — **2,262,940 tokens in total, zero owner mismatches, zero prediction mismatches**. The circuit is the one the checkpoint carries. On the checkpoint itself (§5.4) the same comparison runs during generation: a reference machine advances in lockstep with the model's own tool calls and the run stops at the first token the layers get wrong. Over 700 items — 97,276,073 chip tokens — there were zero such tokens and zero positions where Qwen's head disagreed with the circuit's prediction, on an RTX 4090 in bf16 with the author's Triton executor; the author's `calc` passes on the same machine, and the IR replay of the first demo run is clean (88,309 positions).

The reference machine also serves as the execution environment of the benchmark (§5): the model's code runs on it with the real firmware, so the environment is the in-weights one exactly, simulated at ~300 K instructions/s instead of the executor's ~4 K/s.

4 Cost

Table 1 gives the per-call cost on the reference machine; trace tokens are ≈ 1.25 per instruction; seconds assume the author's executor (~4 K instructions/s on an NVIDIA L4).

Call	Instructions	Trace tokens	≈ L4 s
`pass` (capture, parse, compile, framing)	14.8 K	18 K	4
`print(2**100)`	42.9 K	53 K	11
`s = "…"; count = s.count('r'); print(count)`	79 K	98 K	20
`print(sorted('pear apple fig banana'.split()))`	89 K	111 K	22
`import datetime` + a date sum (once per conversation)	482 K	579 K	120
a `datetime` + `timedelta`, `fromtimestamp`	338 K	421 K	85
`fib(15)`, recursive	1.5 M	1.9 M	480

Table 1. Cost per call. Boot to the first park is 755 instructions, on the host.

Profiling by function showed `mp_parse` at 37–48 % of every call: MicroPython's parser pushes, tries and backtracks out of every alternative of an `or`-rule — `compound_stmt` has nine keyword-led alternatives and `small_stmt` lists `expr_stmt` last of eight — and the same inside expressions. We added FIRST-set pruning: a host-side generator, compiled with the port's exact configuration so rule and token numbering match, computes for every grammar rule the set of tokens that can start it (every token, for nullable rules), and a 31-line patch to `parse.c` skips an alternative, an optional rule or a list item whose rule cannot start with the current token instead of pushing it. The table is 3 KB in the DROM. Instructions per call fell 19–31 % (`pass` 21.6 K → 14.8 K; the `count` call 104 K → 79 K). On the unix port, MicroPython's own suite (`basics`, `float`, `unicode`, `misc`, `micropython`: 701 tests, 23,019 cases) gives identical results with and without the patch, and `compile()` of a typical test file is 34 % faster. The author's design notes mention an unpublished "FIRST-set parse prediction" patch in his build; ours is an independent implementation.

The other large component is allocation (`gc_alloc`/`gc_free`/`memset`, ~20 %). The per-call floor of ~15 K instructions is the price of parsing and compiling even an empty program; it would fall further only with a cached bytecode path.

5 Benchmark

5.1 Setup

Ten tasks with exact answers: six BIG-Bench-Hard subsets (Suzgun et al., 2022) — `word_sorting`, `dyck_languages`, `boolean_expressions`, `date_understanding`, `multistep_arithmetic_two`, `object_counting` — and four synthetic character-level tasks in the style of CUTE (Edman et al., 2024) on nonsense strings built from the BBH vocabulary, so nothing is memorised: `count_char`, `reverse`, `nth_char`, `swap_chars`. Fifty items per task; exact match after task-specific normalisation.

Three modes, same model, same prompts:

- **Base**: no tools; the prompt asks for a final `Answer:` line.
- **Python, in-weights environment**: the `python` tool with the schema of §3; the model's calls are executed on the reference machine with the real firmware, in a fresh session per item; up to four calls, then a final answer turn.
- **Python, host CPython**: the same tool, the same loop, host CPython.

Two runs. The *development* run (§5.2–5.3) used Qwen3.5-4B served by Ollama (Q4_K_M quantisation, thinking disabled, greedy) as a stand-in for the model, with the model's code executed on the reference machine with the real firmware: it measures the in-weights environment with a proxy model and drove the firmware fixes. The *checkpoint* run (§5.4) used `divagr1925/Qwen3.5-4B-Qwenputer` in bf16 on an RTX 4090 with the author's Triton executor (~2,800 chip tokens/s), and stock `Qwen/Qwen3.5-4B` in bf16 as the base, 50 items per task, greedy, thinking disabled.

5.2 Results of the development run (proxy model)

Task	Base	Python, in-weights	Python, host	Calls / item	Overflowing calls	≈ L4 s / item
count_char	68	**100**	100	1.0	0	33
reverse	0	**96**	96	1.0	0	16
nth_char	94	**98**	98	0.6	10	10
word_sorting	60	**98**	98	1.0	0	42
multistep_arithmetic_two	98	**100**	100	0.9	9	16
boolean_expressions	86	**92**	96	1.3	4	23
date_understanding	52	**62**	64	1.2	36	53
swap_chars	24	**64**	74	2.7	60	60
object_counting	86	82	82	0.7	24	6
dyck_languages	26	24	28	1.3	54	10

Task	Base	Python, in-weights	Python, host	Calls / item	Overflowing calls	≈ L4 s / item
mean	**59.4**	**81.6**	**83.6**			

Table 2. Development run: accuracy (%) on 50 items per task with the Ollama Q4 stand-in. "Overflowing calls" are tool calls longer than the 80-token capture window (the firmware returns an error). Cost is per item, including items without a call.

The tool is worth +22 points over the base model. The in-weights column equals host CPython on six tasks and is within 2–4 points on three. The gap on the remaining tasks is where the overflowing calls are: `swap_chars` (60 overflowing calls in 136), `dyck_languages` (54 in 64), `date_understanding` (36), `object_counting` (24). On `dyck` the bracket sequence alone exceeds 80 tokens; and the model cannot write the algorithm even on host (28 %), so the tool does not help a 4B model there at all. `object_counting` is the one task where the tool slightly hurts: the model calls it on fewer than half the items and, when it does, re-emitting the item list overflows.

5.3 Ablation: what the environment cost before the fixes

The first run used the author's stock MicroPython configuration and a first version of the harness. Table 3 compares the in-weights column before and after the changes of §3.2 and §6.

Task	Initial	Final	Cause of the change
reverse	0	96	every call raised `NotImplementedError: only slices with step=1`; the model retried `s[::-1]` five times per item
word_sorting	62	98	91 of 127 calls overflowed: `['thrill', 'splutter', …]` costs 3 tokens a word; the `data` parameter
swap_chars	26	64	163 overflowing calls (comments, verbose code); `data` and the compact-code example
dyck_languages	6	24	overflows; `data`
date_understanding	50	62	`strftime` missing (9 errors); overflows; the harness's final turn
multistep_arithmetic_two	92	100	harness: after four calls the model got no final turn although the tool had printed the answer
count_char	98	100	
boolean_expressions	90	92	
nth_char	94	98	
object_counting	82	82	

Table 3. In-weights accuracy before and after the interpreter and harness changes.

The ablation is the substantive finding of the comparison. An exact executor cannot differ from host CPython on correctly executed code; the 0 % on `reverse` and the 34-point gap on `word_sorting` were environment, not computation, and were invisible until measured.

5.4 Results on the checkpoint

Task	Base bf16	In-weights	In-weights, re-tokenized
count_char	78	**100**	
reverse	0	20	**100**
swap_chars	38	30	**100**

Task	Base bf16	In-weights	In-weights, re-tokenized
word_sorting	72	**96**	90
nth_char	98	98	
multistep_arithmetic_two	100	100	
boolean_expressions	94	92	
object_counting	96	86	
date_understanding	84	54	
dyck_languages	48	6	10
mean	**70.8**	68.2	**83.0**

Table 4. The checkpoint: accuracy (%) on 50 items per task. *Base* is stock Qwen3.5-4B in bf16 with no tool; *in-weights* is the model calling `python`, the chip layers running it and the model printing the response; *re-tokenized* is the same with the driver change of §5.5, run on the four tasks whose answers are strings (the mean takes it where run). Chip tokens generated: 68.98 M (in-weights) and 28.30 M (re-tokenized), zero lock errors and zero head mismatches in both.

Three things differ from the proxy. First, the bf16 base is far stronger than the Q4 stand-in (70.8 vs 59.4 mean; `date_understanding` 84 vs 52, `dyck` 48 vs 26), so the tool's margin is smaller and, on three tasks, negative: `date_understanding` 84 → 54, `object_counting` 96 → 86, `dyck` 48 → 6. These are the tasks whose inputs overflow the window (dates with verbose code, item lists, bracket sequences): the model spends its calls on overflow errors where it would have answered correctly unaided. Second, where the input fits, the picture is the proxy's: `count_char` 100, `multistep` 100, `nth_char` 98, `word_sorting` 96, `boolean` 92. Third, two tasks collapsed on the checkpoint that the proxy had at 96 % and 64 %: `reverse` 20 % and `swap_chars` 30 % — with the machine exact on every token. §5.5 explains them.

5.5 The response is printed one byte per token

The circuit prints a tool response by echoing each `putchar` as the single-character Qwen token of that byte. A response therefore enters Qwen's context tokenized one character at a time, not as the tokenizer would segment it. On `reverse` the tool printed the correct string in 100 % of items and the model's final answer was correct in 20 %: `rotcnufrinevuos` became "rotcnufrinevus", `laitreniibocaj` became "laitreini bocaj". On `swap_chars` the tool was right in 92 % of items and the answer in 30 %. Digits are immune — Qwen tokenizes numbers one digit per token in any case, which is why the author's `calc` results never met this — and single characters and real words largely are (`nth_char` 98 %, `word_sorting` 96 %); a 15-letter nonsense string is the worst case. The Ollama proxy returned tool responses normally tokenized, which is why it showed no loss.

The fix is in the driver, not the firmware (`tools/retok.py`). During a run the echo tokens are withheld from Qwen's layers; when the machine parks, the same response text is tokenized normally and fed to Qwen's layers in their place. The generated stream — what the chip layers consume, what the lockstep reference checks and what the IR replay verifies — is unchanged; the model still prints the response itself; only its representation in Qwen's KV cache differs. `reverse` goes from 20 % to **100 %** and `swap_chars` from 30 % to **100 %**; `word_sorting` 96 → 90 % is five items that overflow the window in both runs; `dyck` stays at the base model's limit. We read this as a design note for in-weights tools generally: a bridge that emits bytes should let the host re-segment them before the language layers read them, or emit the model's own tokens, which the compiled bridge cannot do without the BPE merges.

6 Model behaviour under a token-limited tool

The 80-token window is the binding constraint of general computation in the weights, and the model does not manage it. In the initial runs the model wrote list literals for every word list, padded code with comments, and on receiving the overflow error **resent the identical call** up to four times; instructions in the tool description and system prompt ("no comments", "pass data as `'a b c'.split()`") changed nothing. What

changed the behaviour was the schema: a required `data` parameter described as "the question's data as plain text", plus one example call in the system prompt. With it, `word_sorting` went from 91 overflows in 127 calls to zero, one call per item. Plain text costs ≈ 1.3 tokens a word against 3 for a quoted list element, and the model fills parameters far more reliably than it follows style instructions. We read this as a general point for small models: constraints should be encoded in the interface, not requested in prose.

A second behaviour: when the model exhausts its calls mid-sequence and the tool has already printed the right answer, it needs a turn to say so; removing the tools from the request makes Qwen3.5 return an empty message, so the final turn must be a user-side nudge with the tools still present.

7 Discussion

The comparison is a control. Same model, same prompt, same call format: the in-weights and host columns can only differ through the environment. Their equality where calls fit is therefore the expected result, and its value is attribution — on the proxy it lets the overflow counts be read as the complete explanation of the residual gap, and on the checkpoint it is what isolated the response-tokenization effect: the machine was exact on every token, so the collapse on `reverse` had to be in how the model read the response. The hypothesis-bearing comparison is against the base model: +12 points on the checkpoint with re-tokenization (70.8 → 83.0), concentrated in character-level tasks, and negative on tasks the base already does well whose inputs overflow the window.

Arithmetic is the easy case for the window. A `calc` call is an expression of ~20 tokens and the model writes no algorithm; the author's `calc` column matches his host-tool column to within single problems. General code inverts both: the model must write the algorithm and re-emit the data, and the data alone can exceed the window. Widening the window is a compiler change (a two-level counter in the capture circuit); the cleaner fix would capture the *prompt's* text into the machine, so the model never re-emits data at all — then the data-heavy tasks would look like arithmetic.

Cost. Code in the weights is slow: 6–60 s per item at the author's executor speed, against ~50 ms for a host tool. The trace tokens do not enter the model's context (chip tokens are transparent to Qwen's layers), so the cost is time, not attention. The parser optimisation and the FIRST-set table recover a quarter of it; the remaining floor is parse-and-compile, which a bytecode cache would remove for repeated calls.

What is actually different from a tool. Nothing in accuracy, by construction. What differs is that no runtime exists: the capability ships in the checkpoint and works wherever the architecture runs; execution is matmuls with no system calls to escape to; every step is a token in the stream and can be replayed; and the interpreter's state lives in the model's recurrent state alongside the conversation. Whether those properties are worth 80 tokens and 20 seconds a call is a deployment question, not a benchmark one.

8 Limitations

Fifty items per cell: one item is two points, and several differences in the tables (92 vs 94, 86 vs 90) are inside that. One model size, one GPU. The re-tokenized variant was run on four tasks, not ten. The checkpoint run used the author's Triton executor for the chip layers (the stock HF path gave identical outputs on the demos at a fifth of the speed) and was not re-verified with the IR replay beyond the first demo, although the lockstep reference checked every generated token. The window limit, the ~23 KB heap, MicroPython's remaining differences from CPython (no `json`, `os`, `time`; terser error messages) and the speed (6–60 s per item on a 4090; `import datetime` alone is ~2 minutes) all remain.

9 Related work

Program-aided prompting (PAL, Gao et al., 2022; Program of Thoughts, Chen et al., 2022) and tool learning (Toolformer, Schick et al., 2023) established that letting a model write code and execute it externally removes its arithmetic and symbolic errors; our tool is that paradigm with the executor moved into the forward

pass. Compiled transformers (RASP, Weiss et al., 2021; Tracr, Lindner et al., 2023) and constructions of transformers as programmable computers (Giannou et al., 2023) showed that transformer weights can be written rather than learned; Qwenputer (Agrawal, 2026) does so inside a production model, with the Gated DeltaNet recurrence (Yang et al., 2024) as memory. CUTE (Edman et al., 2024) measures the character-level blindness our string tasks exploit; the circuit sees the call's raw bytes, which is why the tool removes it. BIG-Bench-Hard (Suzgun et al., 2022) supplies the algorithmic tasks. MicroPython (George, 2014–) is the interpreter.

10 Conclusion

A general-purpose interpreter runs exactly inside Qwen3.5-4B's forward passes, driven by the model's own tool calls — 97 M chip tokens on the checkpoint without a wrong one — at a cost of 15–200 K instructions per call that a parser lookahead cuts by a quarter. On ten exact-answer tasks it takes the model from 70.8 % to 83.0 %, turning character-level tasks from near-failure to perfect, and it hurts where the base model is already competent and the call does not fit the circuit's 80-token window. The CPU is never the problem; the window and the byte-level response are, and the second is fixed in the driver.

Appendix: Reproduction

Code, firmware, patches, the benchmark harness and all result files are in the repository. `firmware/build.sh` builds the firmware with brew's `riscv64-elf-gcc` and Debian's picolibc package; `tools/run_ref.py` drives calls on the reference machine; `tools/check_circuit.py` runs the circuit verification; `tools/profile_ref.py` profiles a call; `bench/run_all.sh` and `bench/run_python.sh` run the matrix against Ollama; `bench/recost.py` replays recorded calls on a firmware build; `bench/run_gpu.py` and `gpu/setup_l4.sh` run the real checkpoint on a CUDA GPU.

References

- Agrawal, D. (2026). *Qwenputer: a computer compiled into the weights of a language model.* https://github.com/divagr18/qwenputer
- Chen, W., Ma, X., Wang, X., Cohen, W. W. (2022). Program of Thoughts Prompting. arXiv:2211.12588.
- Edman, L., Schmid, H., Fraser, A. (2024). CUTE: Measuring LLMs' Understanding of Their Tokens. EMNLP 2024.
- Gao, L. et al. (2022). PAL: Program-aided Language Models. arXiv:2211.10435.
- Giannou, A. et al. (2023). Looped Transformers as Programmable Computers. ICML 2023.
- Lindner, D. et al. (2023). Tracr: Compiled Transformers as a Laboratory for Interpretability. NeurIPS 2023.
- Schick, T. et al. (2023). Toolformer. NeurIPS 2023.
- Suzgun, M. et al. (2022). Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. arXiv:2210.09261.
- Weiss, G., Goldberg, Y., Yahav, E. (2021). Thinking Like Transformers. ICML 2021.
- Yang, S. et al. (2024). Gated Delta Networks: Improving Mamba2 with Delta Rule. arXiv:2412.06464.
- Qwen Team (2026). Qwen3.5-4B model card. https://huggingface.co/Qwen/Qwen3.5-4B